DP — FAANG Interview Cheat Sheet

Core rule: State → Last decision → Previous choices → Transition → Optimization

1. DP Array Size

State	Typical size
dp[i] = state at array index i	n
dp[i] = answer for first i elements/characters	n + 1
dp[i][j] = state at grid cell (i,j)	m × n
dp[i] = answer considering values 0...i	maxValue + 1

Shortcut: Index DP → n; Prefix DP → n + 1.

2. How Many Loops?

One loop: dp[i] depends on a constant number of previous states.

```
dp[i] = f(dp[i-1], dp[i-2])
```

Examples: House Robber, Delete and Earn, Decode Ways, Climbing Stairs.

Two loops: for every i, try many possible previous choices.

```
for i → for j < i
```

Examples: $O(n^2)$ LIS, Integer Break, Filling Bookcase Shelves.

Sliding-window transition: if dp[i] needs max/min over j in [i-k, i-1], consider a monotonic deque, heap, or TreeMap.

Examples: Constrained Subsequence Sum, Jump Game VI. Naive O(nk) can become O(n).

3. Common DP Families

Family	Recognition	Examples
Fibonacci DP	dp[i] depends on dp[i-1], dp[i-2]. Often take vs skip.	House Robber, Delete and Earn, Climbing Stairs
Prefix DP	dp[i] describes the first i elements/characters.	Decode Ways, Word Break, Valid Partition
Grid DP	dp[i][j] is the state at a cell; often top/left/diagonal.	Maximal Square, Count Squares, Unique Paths
Subsequence DP	dp[i] often means best subsequence ending at i.	LIS, Constrained Subsequence Sum
Interval Scheduling DP	Take current + best compatible previous OR skip current.	Job Scheduling, interval-removal variants
Knapsack	Take item or don't take item; state often includes capacity.	0/1 Knapsack, Partition Equal Subset Sum
Range / Interval DP	dp[l][r] represents an interval; often try every split k.	MCM, Burst Balloons, Cut Stick

4. How to Derive DP on a New Problem

① **Define the state** — Write exactly what dp[state] means in plain English.

② **Identify the final answer** — Is it dp[n-1], dp[n], max(dp[i]), dp[l][r], etc.?

③ **Find the last decision** — Ask: “What could I have done immediately before reaching this state?”

④ **Count previous choices** — Constant number → usually one loop. All j → often two loops. Window → optimize if possible.

⑤ **Write the recurrence** — Express the current state using previously computed states.

⑥ **Optimize** — Look for monotonic deque, heap, binary search, prefix/suffix tricks, or rolling arrays.

5. DP Decision Tree

New DP problem
↓
What does dp[state] mean?
→ index i → prefix of length i → grid (i,j) → interval [l,r]
↓
What is the last decision?
↓
How many previous choices?
→ constant number → O(n)-style transition
→ all previous j → often $O(n^2)$
→ sliding window → deque / heap / TreeMap
→ all splits k → often interval/range DP

6. Patterns You've Recently Practiced

Problem	Pattern / key insight
Constrained Subsequence Sum / Jump Game VI	Linear DP + monotonic decreasing deque
Delete and Earn	Preprocess by value → House Robber / Fibonacci DP
Decode Ways	Prefix DP; last digit vs last two digits
Maximal Square / Count Squares	Grid DP; same recurrence, different aggregation
Filling Bookcase Shelves	Linear DP; try how many previous books fit on current shelf
Integer Break	Two-loop DP; try every first cut and reuse dp[remaining]

Interview mantra: Don’t ask “Which DP pattern is this?” first. Ask: “What is my state? What is my last decision? Which previous states can lead here?”